

面向 BOOM–Rocket 异构核 协同校验系统的错误传播分析与 屏障回滚机制设计

把“检测到错误”推进为“错误不逃逸、状态可恢复”

问题

- BOOM 先执行，Rocket 后确认
- 副作用可能先于校验发生

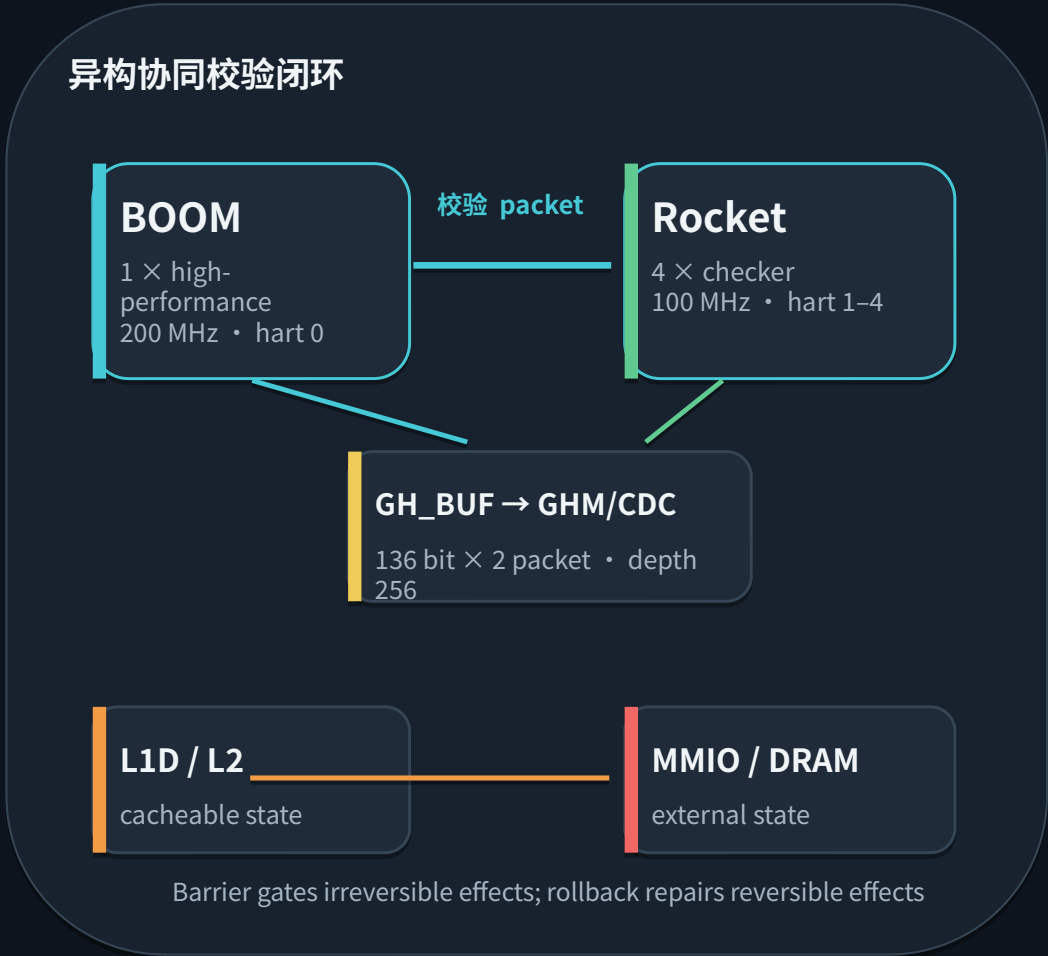
目标

- 隔离未确认状态
- 错误后回到一致点

方法

- 路径分层
- 难度分级

核心抽象： Propagation Path × Rollback Difficulty



为什么需要 Barrier + Rollback ?

核心观点 屏障阻断不可逆传播；回滚撤销已经发生的可逆修改。

错误窗口：执行先于确认



没有 Barrier

- MMIO Put/Atomic 立即生效
- DRAM 写回后难以追回

没有 Rollback

- L1D dirty line 保留错误值
- CSR / reservation 无法重建

设计问题应改写为 3 个追问

01 改写了什么？

store / SC / AMO / CSR / reservation

02 沿哪条路径？

L1D → L2 → DRAM 或 IOMSHR → 外设

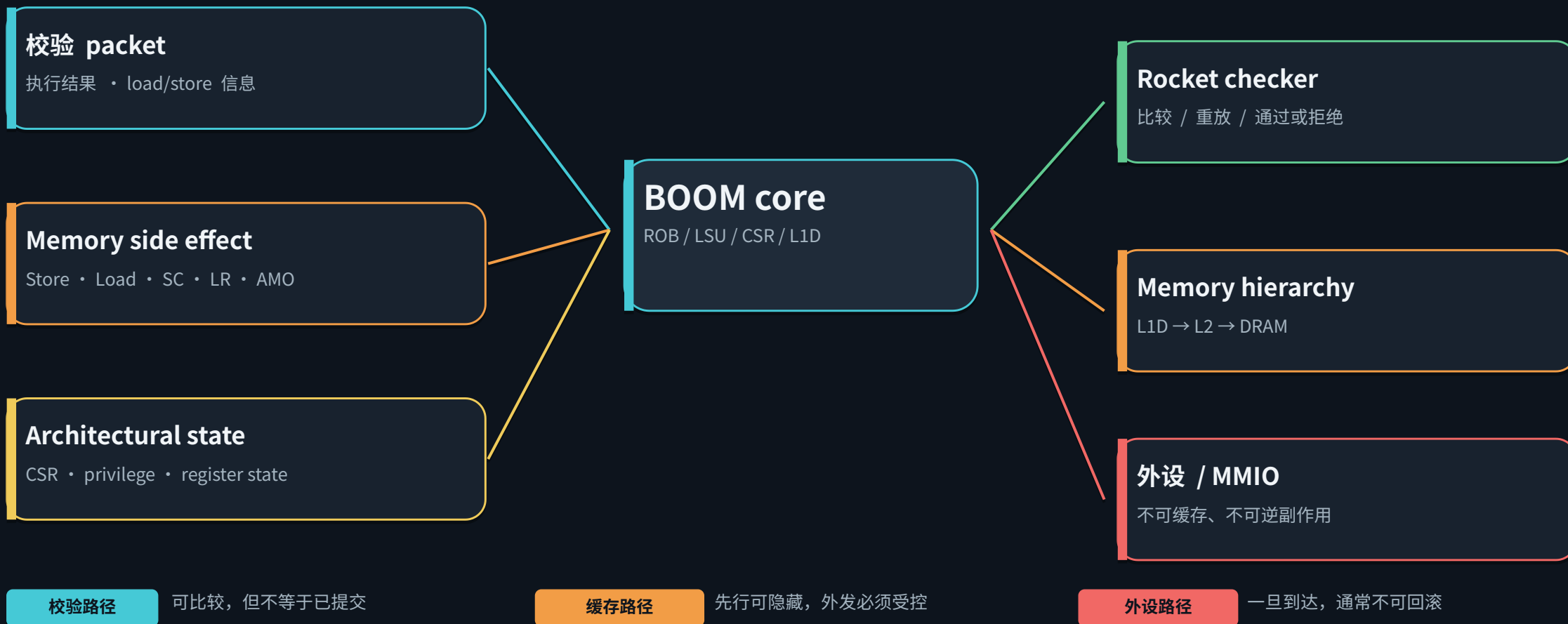
03 在哪里止损？

Barrier before commit + checkpoint / undo

正确性目标：error detected $\neq$ error escaped

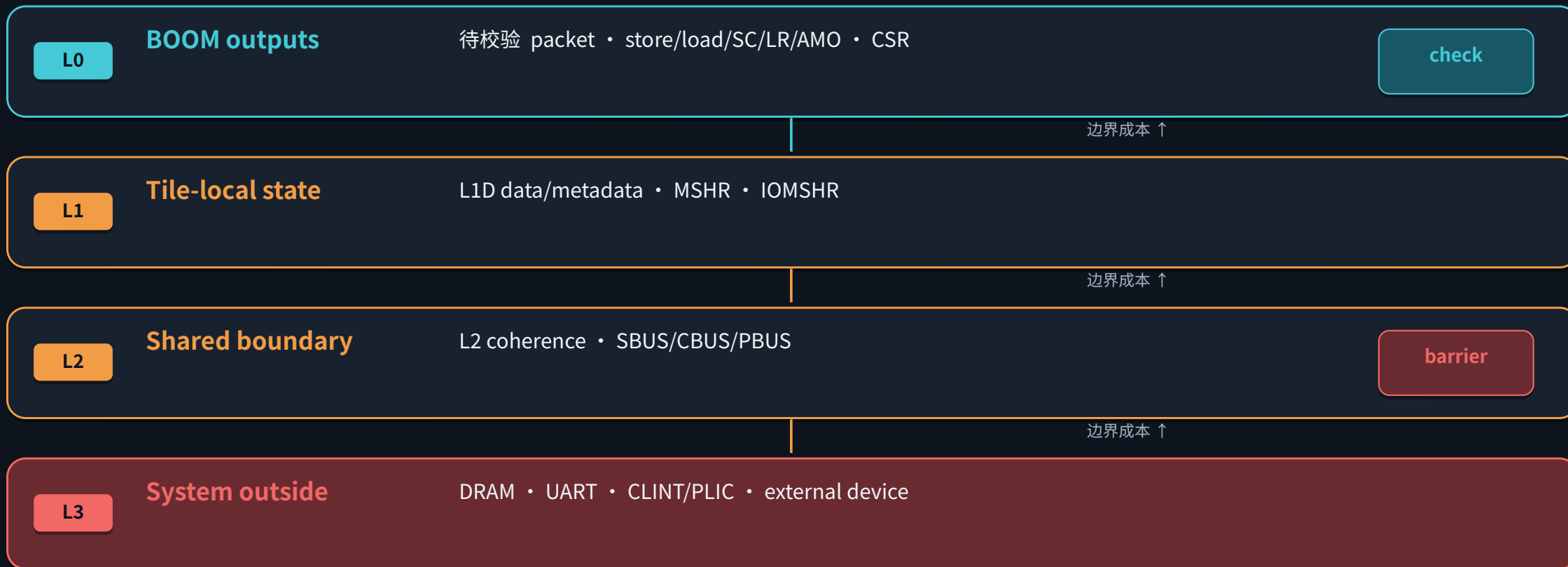
BOOM 执行过程中：哪些状态对外可见？

核心观点 重点不是所有输出，而是能改变系统可观察状态的操作。



错误传播路径：从 BOOM 到外部世界的四层模型

核心观点 每跨过一层边界，恢复成本上升；外设是最外层、最难逆转的屏障。



关键判断点: dmem.req.fire → data-array write → C Release/ProbeAckData → L2 writeback → AXI W/B

Memory Hierarchy 中：同一条 store 如何越过屏障？

核心观点 cacheable 写入可以暂存在层次内部；uncacheable 写入在 A 通道前必须 gate。

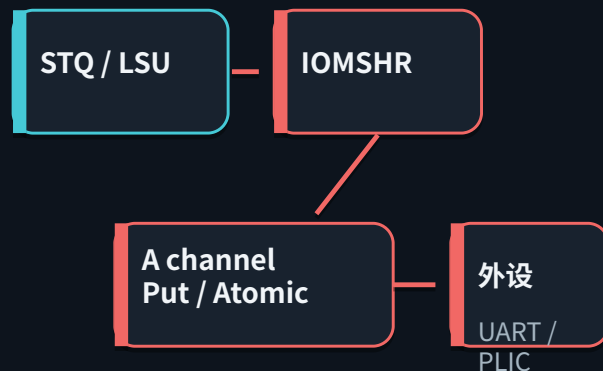
A. Cacheable：可隐藏，但不能任其外发



控制点：line timestamp / epoch + byte mask + eviction / Probe gate

统计口径：`dmem\_req\_fire` = 指令请求；`dataWriteArb.fire` = L1D 真正修改；`tl\_out.c.fire` = cache line 开始外发

B. Uncacheable：外设前的硬屏障



Barrier before commit

等待 Rocket 通过后再发 A

MMIO 一旦被 manager 接收，
core rollback 无法撤回设备副作用

五级恢复难度：从不可提前写出到无影响操作

核心观点 恢复难度由“外部可见性 + 可逆性 + 状态耦合度”共同决定。



边界提醒：普通 load 本身不写外部状态；但 read-clear / FIFO pop / 动态 CSR 属于不可重复输入，需单独记录。

不同类别对应什么恢复策略？

核心观点 策略不是“一刀切暂停”：对不可逆路径加屏障，对可隐藏路径加版本与撤销能力。

状态类别	允许提前做什么	必须记录什么	错误时怎么恢复
uncacheable Store / AMO	不发 A 通道请求	地址、数据、opcode、顺序	Barrier gate；校验通过再 commit
uncacheable Load / MMIO read	可执行一次并缓存返回	返回值、事件序号、side-effect 标记	回放复用，不重复读设备
cacheable Store / SC / AMO	可进入 L1D 私有状态	old value、byte mask、epoch、line metadata	undo log + line restore；禁止外发
CSR / register / privilege	在 speculative window 内推进	architectural checkpoint	恢复 PC + CSR + register state
cacheable Load	自由执行 / 重放	可选的依赖与顺序信息	重新执行或重新校验

统一判据：凡是可能到达不可恢复区域的修改，必须先验证；凡是允许提前执行的修改，必须有 Undo。

整体 BOOM-Rocket 协同校验与恢复框架

核心观点 校验路径决定“何时放行”，状态日志决定“错误后能否回到一致点”。



成功路径: execute → validate → release

错误路径: detect → stop propagation → undo / checkpoint restore → replay

Barrier + Checkpoint + Undo Log：三条设计原则

核心观点 先定位传播路径，再决定屏障；先定义可恢复性，再决定允许多少投机。

01

路径优先

先分析状态传播路径，再决定 Barrier 位置。

`dmem.req.fire` → L1D → C → L2 → AXI / device

02

边界必检

所有可能传播到不可恢复区域的状态修改必须经过校验。

`uncacheable Store / AMO` : Barrier before commit

03

提前必撤销

所有允许提前执行的修改必须具备 Undo 能力。

`checkpoint + old value + epoch / version`

最终目标：BOOM 保持高吞吐，Rocket 提供独立确认；错误不越过不可逆边界，恢复回到一致状态。

下一步验证：MMIO barrier · L1D timestamp/eviction gate · undo log/ECC · checkpoint replay · fault injection